Parallel Web Crawler

FILE 2: Detailed Code Explanation

Every class · Every method · Every parallel concept — explained simply

Contents

Contents 2
1. CrawlerEngine.java — The Heart of the System 3
1.1 Thread-Safe State Variables 3
1.2 start() Method 3
1.3 runParallel() — The Main Parallel Mode 4
1.4 runSequential() — Benchmark Mode (Phase 2) 4
1.5 crawlPageWithRetry() — What Each Worker Thread Does 5
Part 1: Guard Checks 5
Part 2: HTTP Fetch (Stage 2) 5
Part 3: Data Extraction (Stage 3) 5
Part 4: Retry Logic (Phase 2) 5
1.6 BenchmarkResult Record (Phase 3) 6
2. CrawlerConfig.java — Configuration (Builder Pattern) 7
3. CrawledPage.java — Data Model 7
4. Storage Classes 7
4.1 CsvStorage.java 8
4.2 JsonStorage.java 8
4.3 SqliteStorage.java 8
5. MainController.java — GUI Controller 8
5.1 onStart() — When User Clicks START 8
5.2 runCrawl() — Starts Background Thread 8
5.3 onPageResult() — Real-Time GUI Update 9
5.4 displaySpeedupResult() — Phase 3 9
6. How Each Requirement Is Satisfied 9

1. CrawlerEngine.java — The Heart of the System

This is the most important file. It contains ALL the parallel programming logic. It is about 320 lines and is divided into 6 logical sections:

1. State variables (thread-safe data structures)
2. start() — main entry point
3. runSequential() — single-thread mode for benchmarking
4. runParallel() — multi-thread mode (main mode)
5. crawlPageWithRetry() — what each worker thread does
6. saveAndNotify() — stores result and updates GUI

1.1 Thread-Safe State Variables

Declared at the top of the class. Shared by ALL threads:

Variable Name	Type / Value	Why This Type (Thread Safety)
visitedUrls	ConcurrentHashMap.newKeySet()	Atomic add() prevents two threads from queuing the same URL
queue	LinkedBlockingQueue<UrlTask>	Thread-safe FIFO queue — threads block when empty, no manual lock needed
results	Collections.synchronizedList(ArrayList)	Synchronized wrapper prevents concurrent modification exceptions
pageCount	AtomicInteger	CAS-based increment — 10 threads can increment simultaneously without losing counts
stopped	AtomicBoolean	Volatile-like flag — when set to true in onStop(), all worker threads see it immediately
stageDiscoveryMs stageFetchMs stageExtractionMs stageStorageMs	long (Phase 3)	Accumulate milliseconds per stage — used by BenchmarkResult to show per-stage timing

1.2 start() Method

Called by MainController.runCrawl(). Does initial setup then launches the crawl:

```
public List<CrawledPage> start() throws Exception {
    // 1. Reset all state for a fresh crawl
    stopped.set(false);
    pageCount.set(0);
    visitedUrls.clear();
    results.clear();
    stageDiscoveryMs = stageFetchMs = stageExtractionMs = stageStorageMs = 0;

    // 2. Initialize storage based on config flags
```

```
    if (config.isSaveToCSV())     csvStorage    = new CsvStorage(config.getOutputDir());
    if (config.isSaveToJSON())    jsonStorage   = new JsonStorage(config.getOutputDir());
    if (config.isSaveToSQLite()) sqliteStorage = new
SqliteStorage(config.getOutputDir());

    // 3. Add seed URL as the first task (depth=0)
    queue.add(new UrlTask(config.getSeedUrl(), 0));
    visitedUrls.add(config.getSeedUrl());

    // 4. Branch: sequential (benchmark) or parallel (main mode)
    if (config.getThreadCount() == 1) runSequential();
    else                              runParallel();

    // 5. Finalize storage
    if (config.isSaveToJSON())    jsonStorage.finalize(results);
    if (config.isSaveToSQLite()) sqliteStorage.close();
    return results;
}
```

1.3 runParallel() — The Main Parallel Mode

This is where the parallelism happens. Think of it like a factory: the main loop is the manager, and the thread pool is the workers. The manager assigns jobs (URL tasks) to workers, who all work at the same time:

```
private void runParallel() throws Exception {
    // Create N worker threads (N = config.getThreadCount())
    ExecutorService executor = Executors.newFixedThreadPool(config.getThreadCount());

    while (!stopped.get()) {
        if (pageCount.get() >= config.getMaxPages()) break;

        // STAGE 1: Take a batch of URLs from the queue
        List<UrlTask> batch = new ArrayList<>();
        queue.drainTo(batch, config.getThreadCount() * 2);

        if (batch.isEmpty()) {
            Thread.sleep(200); // Wait for new URLs to be discovered
            if (queue.isEmpty()) break; // Nothing left — we're done
            continue;
        }

        // STAGE 2: Submit ALL tasks to the thread pool at once (PARALLEL!)
        List<Future<CrawledPage>> futures = new ArrayList<>();
        for (UrlTask task : batch) {
            futures.add(executor.submit(() -> crawlPageWithRetry(task, 0)));
            // submit() returns immediately — the work happens in background
        }

        // Collect results (wait for all threads to finish this batch)
        List<CrawledPage> batchResults = new ArrayList<>();
        for (Future<CrawledPage> f : futures) {
            CrawledPage page = f.get(timeoutMs + 3000, TimeUnit.MILLISECONDS);
            if (page != null) batchResults.add(page);
        }

        // STAGE 4: Save all results to storage
        for (CrawledPage page : batchResults) saveAndNotify(page);
    }
    executor.shutdownNow();
}
```

1.4 runSequential() — Benchmark Mode (Phase 2)

Same logic as runParallel() but no thread pool — everything runs on one thread, one URL at a time. This is NOT the main mode. It exists ONLY to give us T_sequential for the speedup calculation:

```
private void runSequential() throws Exception {
    // No ExecutorService — just a simple while loop
    while (!stopped.get() && pageCount.get() < config.getMaxPages()) {
        UrlTask task = queue.poll(500, TimeUnit.MILLISECONDS);
        if (task == null) break;

        // Process ONE URL at a time (no parallelism)
        CrawledPage page = crawlPageWithRetry(task, 0);
        if (page != null) saveAndNotify(page);
    }
}
// Total time for this = T_sequential
// Speedup = T_sequential / T_parallel
```

1.5 crawlPageWithRetry() — What Each Worker Thread Does

This method runs inside EACH worker thread. All N threads run this simultaneously on different URLs. It has 4 parts:

Part 1: Guard Checks

```
if (pageCount.get() >= config.getMaxPages() || stopped.get()) return null;
if (task.depth() > config.getMaxDepth()) return null;
// These checks prevent wasted HTTP calls when we're done or stopped
```

Part 2: HTTP Fetch (Stage 2)

```
Thread.sleep(config.getPolitenessDelayMs()); // Be polite to the server
long t0 = System.currentTimeMillis();
Document doc = Jsoup.connect(task.url())
    .userAgent("JavaWebCrawler/1.0")
    .timeout(config.getTimeoutMs())
    .followRedirects(true)
    .get(); // This is where the HTTP request happens
long fetchTime = System.currentTimeMillis() - t0; // How long did it take?
```

Part 3: Data Extraction (Stage 3)

```
String title = doc.title();  // Get page title
String desc = Optional.ofNullable(doc.selectFirst("meta[name=description]"))
    .map(e -> e.attr("content")).orElse("");

List<String> links = new ArrayList<>();
for (Element a : doc.select("a[href]")) {  // Find all links on the page
    String href = a.absUrl("href");          // Get full absolute URL
    if (isValidUrl(href)) {
        links.add(href);
        // ATOMIC DEDUP: only add to queue if URL is NEW
        if (visitedUrls.add(href))  // returns true only if it was new
            queue.offer(new UrlTask(href, task.depth() + 1));
    }
}
```

Part 4: Retry Logic (Phase 2)

```
} catch (IOException e) {
    if (attempt < MAX_RETRIES) {  // MAX_RETRIES = 1
        return crawlPageWithRetry(task, attempt + 1); // Try one more time
    }
    // All retries failed — return an error page
    return new CrawledPage(url, "ERROR", e.getMessage(), emptyList(), 0, 0, depth);
}
```

1.6 BenchmarkResult Record (Phase 3)

A Java record (immutable data object) that holds the timing for each stage. Returned by getLastBenchmark() so MainController can display the numbers:

```
public record BenchmarkResult(
    long discoveryMs,   // Time spent in URL discovery (Stage 1)
    long fetchMs,       // Time spent in HTTP fetch + data extraction (Stage 2+3)
    long extractionMs,  // Time spent extracting links, title, desc
    long storageMs      // Time spent writing to CSV/JSON/SQLite (Stage 4)
) {
    public long totalMs() { return discoveryMs + fetchMs + extractionMs + storageMs; }
}
```

2. CrawlerConfig.java — Configuration (Builder Pattern)

Holds all settings for a crawl. Uses the **Builder design pattern** — instead of one confusing constructor with 10 parameters, you chain readable method calls. Only .build() creates the final object and validates it.

```
// Builder pattern — readable, flexible, validates at the end
CrawlerConfig config = new CrawlerConfig.Builder()
    .seedUrl("https://wikipedia.org")  // Required — crash if missing
    .maxDepth(2)                        // Follow links up to 2 levels deep
    .maxPages(100)                      // Stop after 100 pages total
    .threadCount(6)                     // Use 6 parallel worker threads
    .timeoutMs(10_000)                  // 10 second timeout per HTTP request
    .politenessDelayMs(300)             // Wait 300ms between requests per thread
    .saveToCSV(true)                    // Write CSV output
    .saveToJSON(true)                   // Write JSON output
    .saveToSQLite(true)                 // Write SQLite output
    .outputDir("output")                // Save files in output/ folder
    .build();                           // Validate and create final config
```

Key setting for parallel testing: threadCount controls everything. Set to 1 for sequential benchmark, set to 6 (default) or higher for parallel mode.

3. CrawledPage.java — Data Model

A simple data holder (POJO — Plain Old Java Object) created by each worker thread after successfully crawling a page. Passed to storage writers and to the GUI callback.

Field	Type	What It Contains
url	String	Full URL that was crawled (e.g., https://en.wikipedia.org/wiki/Crawl)
title	String	HTML <title> tag content (e.g., "Web crawler - Wikipedia")
description	String	Content of <meta name="description"> tag — often empty
links	List<String>	All outbound URLs found on this page (already validated via isValidUrl())
statusCode	int	200 = success, 0 = error/timeout/failed-after-retry
crawlTimeMs	long	How long the HTTP request took in milliseconds
crawledAt	LocalDateTime	Exact timestamp when this page was crawled
depth	int	BFS level: 0 = seed, 1 = seed's links, 2 = seed's links' links

4. Storage Classes

Three classes, all implementing the same StorageWriter interface. Each has a **synchronized save(CrawledPage page)** method so multiple threads can call it safely.

4.1 CsvStorage.java

Opens a PrintWriter to crawled_pages.csv. Writes one comma-separated row per page. Header row: URL, Title, Description, LinkCount, StatusCode, CrawlTimeMs, Depth, CrawledAt.

Why synchronized: If two threads write to the PrintWriter at the same time without synchronization, the text from both writes could be interleaved — producing corrupted rows.

4.2 JsonStorage.java

Writes NDJSON (Newline-Delimited JSON) — one complete JSON object per line. This format is great for streaming and log processing.

finalize() method: Called at the end of crawling. Writes a separate crawled_pages_summary.json with aggregate stats: total pages, success count, error count, total links found.

4.3 SqliteStorage.java

Creates a relational database with 2 tables:

- **pages table:** one row per crawled page (url, title, desc, status_code, crawl_time, depth, crawled_at)
- **links table:** one row per link found (source_url, target_url) — creates a link graph

```
-- Useful queries after crawling:
SELECT url, title, status_code, depth FROM pages ORDER BY depth;
SELECT source_url, COUNT(*) as links_found FROM links GROUP BY source_url;
SELECT depth, COUNT(*) as pages_at_depth FROM pages GROUP BY depth;
```

Uses batch inserts (addBatch/executeBatch) and transactions (setAutoCommit(false)) for performance.

5. MainController.java — GUI Controller

Controls the JavaFX window. Handles all button clicks, updates all UI elements in real time, and implements benchmark mode.

5.1 onStart() — When User Clicks START

Validates the URL input, builds a CrawlerConfig, checks if benchmark mode is enabled. If benchmark mode is ON: calls runCrawl() twice (first with threadCount=1 for sequential, then with the configured thread count for parallel).

5.2 runCrawl() — Starts Background Thread

Creates a CrawlerEngine, sets the two callbacks, starts a **daemon thread** to run the crawl in the background (so the GUI stays responsive), and launches an AnimationTimer to update the elapsed time and progress bar every screen frame.

5.3 onPageResult() — Real-Time GUI Update

Called by CrawlerEngine from a worker thread whenever a page is crawled. Uses Platform.runLater() to safely schedule GUI updates. Appends a log line, adds a row to the table, updates the stats counters.

5.4 displaySpeedupResult() — Phase 3

Calculates and displays the benchmark results:

```
double speedup = (double) seqTotalMs / parTotalMs;
double efficiency = (speedup / threadsSpinner.getValue()) * 100;

// Example log output:
// BENCHMARK RESULTS:
//   Sequential: 58,400 ms
//   Parallel (6 threads): 13,200 ms
//   Speedup: 4.42x  (Efficiency: 73.7%)
```

6. How Each Requirement Is Satisfied

Requirement	How We Implement It	Code Location
Parallel downloading	ExecutorService submits N tasks simultaneously	CrawlerEngine.runParallel() — executor.submit()
Parallel extraction	Each thread independently calls doc.title(), select()	CrawlerEngine.crawlPageWithRetry() — Stage 3
Parallel storage	synchronized save() in all 3 storage classes	CsvStorage / JsonStorage / SqliteStorage
Error handling	crawlPageWithRetry retries once on IOException	CrawlerEngine.crawlPageWithRetry() catch block
No duplicate crawls	visitedUrls.add() is atomic — ConcurrentHashMap	CrawlerEngine — visitedUrls field
Graceful stop	stopped.set(true) → all threads check stopped.get()	MainController.onStop() → CrawlerEngine.stop()
Performance comparison	runSequential() then runParallel() → Speedup = T_seq/T_par	MainController.displaySpeedupResult()
Live dashboard	Platform.runLater() updates GUI from background threads	MainController.onPageResult()
Multi-format storage	Three separate synchronized writers run after each batch	CrawlerEngine.saveAndNotify()